

Research Assistant

A Local-First Research Workflow For Papers, Code, and Technical Writing

Colleague Adoption Proposal

April 29, 2026

Contents

1	Executive Summary	1
1.1	Current release scope	1
1.2	Why a colleague should care	2
2	The Concrete Workflow Problem	3
3	What The Package Does In The Current Release	4
3.1	Core capabilities	4
3.2	What the package promises	4
3.3	Trust boundary	5
4	Why Local-First Is The Right Design	6
5	How It Fits Into Current Assistant Workflows	7
5.1	Claude Code and terminal workflows	7
5.2	Why this is better than assistant-only memory	7
5.3	What the package does not replace	7
6	Concrete Example Workflows	8
6.1	Example 1: Ingest a paper and inspect what is safe to trust	8
6.2	Example 2: Expand nearby literature without losing the evidence trail	8
6.3	Example 3: Export trusted context for writing or coding	9
6.4	Nontrivial showcase: DSGE survey chapter from a NeuTra seed	9
7	Scope Boundaries And Non-Goals	11
7.1	Current boundaries	11
7.2	Non-goals for the current release	11
7.3	Known limits that matter to adoption	11
8	Technical Credibility Without The Monograph	12
8.1	Structured local artifacts and provenance	12
8.2	Source-first audit posture	12
8.3	Explicit parser limits	12
8.4	Complements existing coding assistants	13

9	A Practical Adoption Path	14
9.1	Start from a wheel or local checkout	14
9.2	Run the demo before trusting a real workflow	14
9.3	A release-proven use case: source-first NeuTra audit	14
9.4	Use support commands safely	15
10	Skeptical Questions And Straight Answers	16
10.1	Why not just keep using a PDF reader and a note folder?	16
10.2	Why not just use Claude Code directly?	16
10.3	Is this trying to automate research judgment?	16
10.4	Is this already a shared research platform?	16
11	Future Extensions	17
A	Operational Notes Kept Briefly	18
A.1	Git sharing remains conservative	18
A.2	Release maturity remains pilot-scoped	18
	Closing Note	19

Chapter 1

Executive Summary

‘research-assistant’ is a local-first research development package for people who regularly move between papers, notes, code, and technical writing. It is built for one serious researcher first: someone who wants a private, inspectable, file-based workspace rather than a shared hosted platform.

The current release focuses on a bounded but useful problem. It helps a researcher ingest papers, inspect source and parser evidence, preserve provenance and review state, discover nearby work, export trusted context for writing or coding, and selectively share reviewable artifacts through Git. It is not a shared department server, not a database-backed collaboration product, not a hosted UI, and not a default live-provider workflow.

The package is appealing precisely because it is modest in scope. A colleague can install it locally, initialize a workspace, ingest a paper, inspect the result, and decide what to trust without depending on a service operator or central infrastructure. The local-first design keeps the research trail visible, backups straightforward, and failure modes easier to understand.

The adoption argument is therefore practical rather than grand. The tool does not replace judgment, and it does not pretend that machine extraction is proof. Instead, it reduces context loss and manual literature bookkeeping for people who already do serious technical reading and implementation work.

1.1 Current release scope

The current release target is:

- one local researcher first;
- local filesystem storage;
- offline/provider-disabled default workflows;
- Git-based sharing by repository exchange and explicit merge/import;
- explicit review status and provenance rather than silent trust.

The package deliberately does *not* promise:

- shared backend infrastructure;
- shared database writes;
- hosted UI, SSO/RBAC, or real-time collaboration;
- parser scientific accuracy certification;

- automatic promotion of generated artifacts into accepted conclusions.

1.2 Why a colleague should care

A technically skeptical colleague usually does not need another generic paper manager or another generic AI interface. The useful gap is narrower: preserving an auditable bridge between literature reading and local implementation work.

That gap shows up in familiar ways:

- papers are downloaded, skimmed, and then lost in folders;
- context about why a paper mattered disappears between coding sessions;
- implementation decisions drift away from the literature that motivated them;
- later writing requires reconstructing claims, caveats, and nearby work from memory;
- assistant conversations produce useful draft thinking but not durable, inspectable research state.

‘research-assistant’ helps by giving those activities a local, structured, and reviewable home.

Chapter 2

The Concrete Workflow Problem

A serious research workflow is usually not blocked by a lack of PDFs. It is blocked by weak continuity between reading, implementation, and writing.

A researcher may begin with a seed paper, inspect code, try a derivation, follow some citations, and eventually draft a note or chapter. Each step is reasonable. The problem is that the evidence trail often fragments:

- paper metadata lives in a filename or browser tab;
- extracted text or notes live somewhere else;
- coding decisions live in comments or memory;
- literature relevance is rediscovered from scratch during later writing;
- uncertainty is hidden rather than recorded explicitly.

Existing tools usually solve only part of the problem. A PDF reader is good for reading but not for paper-to-code traceability. A coding assistant is good at local code context but does not automatically maintain a durable literature record. A note-taking workflow can hold conclusions, but often loses provenance and review status.

The package is designed to close that narrower gap. It is not trying to replace every tool. It is trying to create a reliable local workflow for moving from paper ingestion to inspectable evidence to trusted downstream context.

Chapter 3

What The Package Does In The Current Release

The current package provides a conservative ingest, review, discover, and export workflow.

3.1 Core capabilities

The implemented release supports:

- local workspace initialization, validation, repair, and configuration;
- paper ingestion from arXiv IDs, local PDFs, and identifying queries;
- structured-source-first inspection when arXiv LaTeX source is available;
- parser readiness checks, parser capability-limit reporting, and parser-benchmark smoke checks;
- explicit review state for local paper records;
- discovery of related, citing, and cited work through scholarly APIs when available;
- duplicate-aware download proposals into a local inbox;
- trusted-context export for downstream writing and coding workflows;
- backup, restore, privacy-status, and release-report workflows;
- Git-sharing hygiene, dry-run merge, explicit apply, and post-merge rebuild.

3.2 What the package promises

The package is strongest where its promises are deliberately narrow:

- structured-source-first for arXiv papers when source is available;
- local-first and file-based behavior;
- conservative handling of uncertainty;
- clear provenance and review status;
- explicit parser capability limits rather than implied reliability;
- remote enrichment that degrades visibly instead of blocking local work;
- no silent final moves for downloaded papers.

3.3 Trust boundary

Generated outputs are review material. Parser outputs, benchmark results, derivation worksheets, experiment records, synthesis proposals, traceability reports, readiness reports, and validation records do not certify mathematical correctness, scientific parser accuracy, or research approval.

That boundary is not a disclaimer attached at the end. It is the core product posture. The value of the package is that it helps a researcher keep evidence, provenance, and uncertainty visible while human judgment remains explicit.

Chapter 4

Why Local-First Is The Right Design

The current release is intentionally local-first and file-based so it remains inspectable and easy to debug.

That design has practical advantages for the intended user:

- the workspace can be inspected with ordinary filesystem tools;
- backups are simple and explicit;
- failure modes are easier to reason about than in opaque hosted systems;
- the package can be useful without depending on central infrastructure;
- a user can choose when and how to share artifacts rather than being forced into a default shared state.

For this release, Git is the sharing layer rather than a transactional database. A researcher can publish or exchange a repository that contains only shareable artifacts, another researcher can inspect it, run repository hygiene checks, dry-run a workspace merge, and apply a safe subset only with explicit confirmation.

This makes collaboration possible without pretending that Git solves semantic research disagreement. Merge conflicts and accepted-audit conflicts remain human review events.

Chapter 5

How It Fits Into Current Assistant Workflows

The package is best understood as a local research memory and evidence layer that complements coding assistants.

5.1 Claude Code and terminal workflows

The strongest fit today is a researcher who already uses Claude Code or ordinary terminal workflows. In that setting, the package provides durable local context: structured summaries, source records, review state, discovery output, trusted exports, and linkage artifacts that survive beyond a single assistant session.

5.2 Why this is better than assistant-only memory

Assistant conversations are helpful, but they are not automatically the canonical research record. A local package helps when the user wants:

- inspectable artifacts rather than hidden state;
- repeatable commands rather than only conversational recall;
- provenance and explicit review status;
- exports that can be reused for later coding or writing.

5.3 What the package does not replace

The package is not a replacement for Claude Code, Cursor, Copilot, or a PDF reader. It fills the space between them by preserving structured local research state that those tools can then consume more effectively.

Chapter 6

Concrete Example Workflows

6.1 Example 1: Ingest a paper and inspect what is safe to trust

A common first-use case is simple: a researcher has a paper that may matter for a project, but does not want to trust a vague memory or a one-shot summary.

The workflow can begin with a local workspace and a single ingest:

```
ra --root ~/ra-workspace init
ra --root ~/ra-workspace ingest \
  --pdf ~/papers/example.pdf \
  --query "paper title"
ra --root ~/ra-workspace show --paper-id paper_example
ra --root ~/ra-workspace review-show --paper-id paper_example
```

The important point is not that the first output is magically final. The useful behavior is that the researcher can inspect:

- metadata provenance;
- parser/source extraction status;
- confidence and review state;
- what still requires human review.

This is a better starting point than a disconnected PDF folder because the uncertainty is visible rather than hidden.

6.2 Example 2: Expand nearby literature without losing the evidence trail

Suppose the researcher has found one promising seed paper and wants a fast but reviewable picture of nearby work.

```
ra --root ~/ra-workspace find --query "Neural Transport HMC"
ra --root ~/ra-workspace citation-neighborhood \
  --paper-id neutra_hmc
ra --root ~/ra-workspace discover --query "transport maps hmc"
ra --root ~/ra-workspace inbox-list
```

This does not turn literature discovery into an automatic truth engine. It helps a researcher see what is nearby, what is available, and what needs explicit inspection. Remote enrichment may be empty or unavailable; the package surfaces that status rather than pretending success.

That behavior is useful for real work because it reduces manual graph traversal while keeping the review boundary intact.

6.3 Example 3: Export trusted context for writing or coding

A later pain point is not finding papers, but recovering trusted local context for a coding session or technical note.

```
ra --root ~/ra-workspace review-mark \  
  --paper-id neutra_hmc \  
  --status approved  
ra --root ~/ra-workspace export-context \  
  --review-status approved \  
  --output /tmp/paper_context.json
```

The value here is practical. Once the researcher has approved a subset of local records, the package can produce a conservative context export for downstream use in writing or coding tools.

That export is intentionally bounded. It should come from reviewed records, not from everything that happened to be ingested. The package therefore helps with speed *and* discipline.

6.4 Nontrivial showcase: DSGE survey chapter from a NeuTra seed

A more persuasive use case is a DSGE survey chapter from a NeuTra seed. Suppose a researcher wants to compare normalizing-flow and neural-transport architectures for difficult posterior geometry. The package does not replace the researcher or write the survey automatically. It gives the researcher a durable workspace for seed papers, citation neighborhoods, architecture taxonomies, chapter links, code experiments, review notes, and trusted context exports.

The strongest current version of this workflow is source-first when the seed paper is known and available on arXiv. In that setting, the package is most useful not as an automatic literature engine, but as a local evidence layer that lets the researcher inspect what the paper explicitly says before updating code, notes, or chapter prose.

```
ra --root ~/ra-workspace ingest \  
  --query "NeuTra-lizing Bad Geometry in Hamiltonian Monte Carlo Using Neural  
    Transport"  
ra --root ~/ra-workspace ingest \  
  --query "Normalizing Flows for Probabilistic Modeling and Inference"  
ra --root ~/ra-workspace citation-neighborhood \  
  --paper-id neutra_hmc --limit 25  
ra --root ~/ra-workspace synthesis propose \  
  --paper-id neutra_hmc --kind survey_chapter_outline
```

```
ra --root ~/ra-workspace traceability build --paper-id neutra_hmc
ra --root ~/ra-workspace export-context \
  --review-status approved \
  --output /tmp/dsge_flow_chapter_context.json
```

This broad showcase is useful for chapter planning and evidence organization. But for mathematically sensitive material, the higher-trust workflow is often to fetch and inspect source directly, then treat discovery and synthesis as secondary, reviewable aids.

Chapter 7

Scope Boundaries And Non-Goals

A colleague should adopt the package for what it already does well, not for what it might become later.

7.1 Current boundaries

This release is:

- private local use rather than shared departmental deployment;
- Git sharing by repository or snapshot rather than real-time collaboration;
- offline/provider-disabled by default;
- strongest on local auditability, provenance, and trusted-context export.

7.2 Non-goals for the current release

This release does not aim to provide:

- a GUI-first rewrite;
- a database requirement;
- bulk scraping or broad web crawling;
- silent auto-organization of a paper library;
- high-stakes claim verification without explicit evidence support.

7.3 Known limits that matter to adoption

Colleagues should know the practical limits up front:

- parser quality depends on local optional tools and source/PDF quality;
- parser-tool availability can be checked, but parser scientific accuracy is not certified;
- Git-sharing performance evidence is currently synthetic at the ‘synthetic_git_1000’ tier unless a non-sensitive real corpus is explicitly recorded;
- real colleague onboarding, real macOS validation, and real minimal parser-tool validation remain manual release gates.

Chapter 8

Technical Credibility Without The Monograph

The package is persuasive only if technically serious readers believe the design is disciplined.

8.1 Structured local artifacts and provenance

The workspace keeps local records for metadata, summaries, reviews, source artifacts, downloaded-paper proposals, and exportable trusted context. This is not just a storage choice. It means the user can inspect where information came from and what status it currently has.

8.2 Source-first audit posture

For arXiv papers, LaTeX source is preferred when available. That is a strong and practical design choice for mathematical material because source often provides a better audit substrate than PDF parsing alone. The package therefore treats PDF parser output as fallback and cross-check material rather than as a silently trusted primary source.

In practice, a good rule is: if the paper is known and source is available, use ‘source-fetch’ first when the task is chapter audit, derivation checking, code-literature alignment, or equation-level review. That workflow is especially useful when the researcher needs to verify what a paper explicitly claims before rewriting technical documentation or implementation notes.

8.3 Explicit parser limits

The package is careful about parser claims. Commands such as ‘ra doctor –matrix’, ‘ra parser-tool-matrix’, and ‘ra parser-benchmark-smoke’ exist to surface tool availability, workflow readiness, fixture smoke behavior, and capability limits. That is more trustworthy than pretending equations, citations, or section structure are always reliable.

8.4 Complements existing coding assistants

The package becomes more useful when paired with existing assistant workflows. Its role is to preserve durable local research state — reviewed paper records, source-derived evidence, discovery results, and trusted exports — that an assistant or script can use later.

Chapter 9

A Practical Adoption Path

A colleague does not need to commit to a department-wide workflow to evaluate the package. The sensible adoption path is a small local pilot.

9.1 Start from a wheel or local checkout

```
python -m pip install research_assistant-0.1.0-py3-none-any.whl
ra version
ra --root ~/ra-workspace init
ra --root ~/ra-workspace doctor
```

A developer can also install from a checkout with ‘python -m pip install .’, but the colleague-facing path is the wheel and first-workspace flow.

9.2 Run the demo before trusting a real workflow

```
ra --root /tmp/research-assistant-demo demo setup
ra --root /tmp/research-assistant-demo demo run
ra --root /tmp/research-assistant-demo release-report
```

This gives the user a deterministic, non-sensitive first experience before any real paper is involved.

9.3 A release-proven use case: source-first NeuTra audit

The release was exercised on a realistic research task: auditing a DSGE/HMC monograph chapter built around *NeuTra-lizing Bad Geometry in Hamiltonian Monte Carlo Using Neural Transport*. This is a good colleague-facing example because it shows where the package is already genuinely useful: a known-paper audit with source-grounded inspection and downstream writing support.

Representative commands:

```
ra --root /tmp/ra-neutra-audit init
ra --root /tmp/ra-neutra-audit ingest --arxiv-id 1903.03704 \
```

```
--query "NeuTra-lizing Bad Geometry in Hamiltonian Monte Carlo Using Neural
Transport"
ra --root /tmp/ra-neutra-audit source-fetch --arxiv-id 1903.03704
ra --root /tmp/ra-neutra-audit find --query "NeuTra"
ra --root /tmp/ra-neutra-audit source-sections --paper-id paper_arxiv_1903_aa33312a
ra --root /tmp/ra-neutra-audit source-equations --paper-id paper_arxiv_1903_aa33312a
ra --root /tmp/ra-neutra-audit source-show --paper-id paper_arxiv_1903_aa33312a
```

What this illustrates in practice:

- source-first inspection is already useful for mathematical literature audits;
- structured source outputs make chapter and code verification easier than PDF-only review;
- reviewed context can then be exported into later writing or coding workflows.

What it does *not* illustrate:

- automatic survey writing;
- automatic mathematical certification;
- reliable open-ended literature discovery without human review.

This is exactly the level at which the current release should be presented to colleagues: strong for local, source-grounded audits of known papers, and honest about where human judgment still dominates.

9.4 Use support commands safely

If something fails, the support boundary is explicit: share diagnostics, not research data.

Useful commands include:

```
ra version
ra doctor --matrix
ra platform-status
ra privacy status
ra parser-tool-matrix
```

This is another adoption advantage of the local-first posture: diagnosis can be performed on a demo or empty workspace without exposing actual research content.

Chapter 10

Skeptical Questions And Straight Answers

10.1 Why not just keep using a PDF reader and a note folder?

Because the real cost is not reading alone. The cost is reconstructing later why a paper mattered, what was trusted, what was only provisional, and how the paper related to code or writing. A local structured workflow is useful when that continuity matters.

10.2 Why not just use Claude Code directly?

Claude Code is very useful for code and local context, but it does not by itself become the durable research record. The package gives the user a stable local workspace of summaries, provenance, review state, source artifacts, and trusted exports that can then be used in Claude Code more effectively.

10.3 Is this trying to automate research judgment?

No. The package is deliberately conservative. Generated and parser-derived artifacts remain review material, and human approval remains explicit.

10.4 Is this already a shared research platform?

No. The current release is intentionally smaller and more honest than that. It is a local-first individual workflow with Git-based sharing, not a hosted multi-user system.

Chapter 11

Future Extensions

Future work may extend the package, but those possibilities should not be confused with the current adoption case.

Possible later directions include:

- stronger parser and source benchmarks on gold corpora;
- richer sanitized real-corpus scalability evidence;
- optional governed provider workflows;
- more developed team conventions around artifact exchange.

Those are not prerequisites for the current package to be useful. The present value proposition is that the local tool is already inspectable, bounded, and practically helpful.

Appendix A

Operational Notes Kept Briefly

A.1 Git sharing remains conservative

Before sharing a repository, a researcher should run:

```
ra repository-hygiene check --strict
ra workspace merge \
  --source /path/to/other/repo \
  --target ~/ra-workspace
ra workspace merge \
  --source /path/to/other/repo \
  --target ~/ra-workspace \
  --apply --confirm-merge
ra workspace rebuild-derived
```

Dry-run is the default. Apply requires explicit confirmation. Accepted-audit conflicts remain human review events.

A.2 Release maturity remains pilot-scoped

Broad release still requires real fresh-reader onboarding, real macOS validation, real minimal parser-tool machine validation, release-owner tag approval, and release-owner publication approval. Until those are recorded, the release remains pilot-scoped.

Closing Note

The honest reason to adopt ‘research-assistant’ is not that it promises a future research platform. It is that, today, it offers a private, local, inspectable, and conservative workflow for carrying paper context into coding and writing work without losing provenance or review discipline.